STUDENT MANAGEMENT SYSTEM

JDBC Project – Step-by-Step Student Guide

Technology: Java + JDBC + MySQL

Architecture: DTO + DAO + DAO Implementation + ConnectionFactory

Database: MySQL | **Tables:** 2 only | **JOIN:** Not required

1. Project Objective

Build a menu-driven Student Management System using JDBC. The project should demonstrate database connectivity, CRUD operations, DAO/DTO design, PreparedStatement, ResultSet, search operations and basic SQL conditions.

2. Concepts You Must Use

Concept	Requirement
Java	Core Java, classes, objects, Scanner, loops, switch
JDBC	Connection, PreparedStatement, ResultSet
DAO	Separate database operations from main program
DTO	Store and transfer data using Java objects
ConnectionFactory	Create/reuse database connections
MySQL	Create and manage the database and tables
SQL	INSERT, SELECT, UPDATE, DELETE, WHERE, LIKE, BETWEEN, ORDER BY, COUNT

3. Project Package Structure

Create the project using the following package structure. Do not put all JDBC code inside the main class.

```
StudentManagementSystem ■ ■■■ connection ■ ■■■ ConnectionFactory.java ■ ■■■ dto ■ ■■■ Student.java ■ ■■■ Course.java ■ ■■■ dao ■ ■■■ StudentDAO.java ■ ■■■ CourseDAO.java ■ ■■■ daoimpl ■ ■■■ StudentDAOImpl.java ■ ■■■ CourseDAOImpl.java ■ ■■■ main ■■■ StudentManagement.java
```

4. Step 1 – Create the MySQL Database

Create the database first.

```
CREATE DATABASE student_db; USE student_db;
```

5. Step 2 – Create the Student Table

```
CREATE TABLE student ( student_id INT PRIMARY KEY AUTO_INCREMENT, name VARCHAR(100), email VARCHAR(100), phone VARCHAR(15), age INT );
```

Student table fields

Column	Data Type	Purpose
student_id	INT	Unique student ID
name	VARCHAR(100)	Student name
email	VARCHAR(100)	Student email
phone	VARCHAR(15)	Phone number
age	INT	Student age

6. Step 3 – Create the Course Table

```
CREATE TABLE course ( course_id INT PRIMARY KEY AUTO_INCREMENT, course_name VARCHAR(100), duration VARCHAR(50), fees DOUBLE );
```

Column	Data Type	Purpose
course_id	INT	Unique course ID
course_name	VARCHAR(100)	Course name
duration	VARCHAR(50)	Course duration
fees	DOUBLE	Course fees

7. Step 4 – Insert Sample Data

Insert at least 5 students and 5 courses manually in MySQL. This will help you test SELECT and search operations.

```
INSERT INTO student(name,email,phone,age) VALUES ('Rahul','rahul@gmail.com','9876543210',22);
INSERT INTO course(course_name,duration,fees) VALUES ('Java Full Stack','6 Months',35000);
```

8. Step 5 – Add the MySQL JDBC Driver

Add the MySQL Connector/J dependency or JAR to your Java project. Make sure the JDBC driver is available before testing ConnectionFactory.

9. Step 6 – Create ConnectionFactory

Create one common class responsible for establishing the database connection. Replace the username and password with your own MySQL credentials.

```
package connection; import java.sql.Connection; import java.sql.DriverManager; public class ConnectionFactory { private static final String URL = "jdbc:mysql://localhost:3306/student_db"; private static final String USER = "root"; private static final String PASSWORD = "your_password"; public static Connection getConnection() { try { return DriverManager.getConnection(URL, USER, PASSWORD); } catch (Exception e) { e.printStackTrace(); return null; } } }
```

Checkpoint 1

Before continuing, successfully connect Java to MySQL using ConnectionFactory.

10. Step 7 – Create Student DTO

Create a Student class containing private fields, constructors, getters, setters and toString().

```
private int studentId; private String name; private String email; private String phone; private int age;
```

11. Step 8 – Create Course DTO

```
private int courseId; private String courseName; private String duration; private double fees;
```

Checkpoint 2

You should now have Student.java and Course.java. Test the DTO classes by creating objects and printing their values.

12. Step 9 – Create StudentDAO Interface

Define the database operations without writing SQL in the interface.

```
public interface StudentDAO { void addStudent(Student student); List getAllStudents(); Student getStudentById(int id); void updateStudent(Student student); void deleteStudent(int id); List searchByName(String name); List searchByAgeRange(int minAge, int maxAge); int countStudents(); }
```

13. Step 10 – Create CourseDAO Interface

```
public interface CourseDAO { void addCourse(Course course); List getAllCourses(); Course getCourseById(int id); void updateCourse(Course course); void deleteCourse(int id); List searchByName(String name); List searchByDuration(String duration); List searchByFeeRange(double min, double max); int countCourses(); }
```

14. Step 11 – Implement StudentDAOImpl

Create StudentDAOImpl and implement every method from StudentDAO. Use ConnectionFactory for the connection.

Rules:

• Use PreparedStatement for SQL with input values.

• Use ResultSet for SELECT operations.

• Use executeUpdate() for INSERT, UPDATE and DELETE.

• Use executeQuery() for SELECT.

• Do not write database code inside StudentManagement.java.

• Use try-with-resources where possible.

15. Step 12 – Implement CourseDAOImpl

Create CourseDAOImpl and implement all CourseDAO methods using the same JDBC approach.

16. Step 13 – Test Student CRUD

Task	SQL concept
Add Student	INSERT
View All Students	SELECT
Find Student by ID	SELECT + WHERE
Update Student	UPDATE + WHERE
Delete Student	DELETE + WHERE

17. Step 14 – Test Course CRUD

Task	SQL concept
Add Course	INSERT
View All Courses	SELECT
Find Course by ID	SELECT + WHERE
Update Course	UPDATE + WHERE
Delete Course	DELETE + WHERE

18. Step 15 – Add Search Features

• Search students by name using LIKE.

• Search students by age range using BETWEEN.

• Search courses by name using LIKE.

• Search courses by duration.

• Search courses by fee range using BETWEEN.

• Display students sorted by name using ORDER BY.

• Display courses sorted by fees using ORDER BY.

Example SQL

```
SELECT * FROM student WHERE name LIKE ?; SELECT * FROM student WHERE age BETWEEN ? AND ?; SELECT * FROM course WHERE fees BETWEEN ? AND ?; SELECT * FROM student ORDER BY name;
```

19. Step 16 – Add Count Operations

```
SELECT COUNT(*) FROM student; SELECT COUNT(*) FROM course;
```

20. Step 17 – Create the Main Menu

Create StudentManagement.java. The main class should handle user input and call DAO methods. It should not contain SQL queries.

```
====================================== STUDENT MANAGEMENT SYSTEM
====================================== STUDENT 1. Add Student 2. View All Students 3. Find Student 4. Update Student 5. Delete Student 6. Search Student by Name 7. Search Student by Age COURSE 8. Add Course 9. View All Courses 10. Find Course 11. Update Course 12. Delete Course 13. Search Course by Name 14. Search Course by Fee REPORT 15. Count Students 16. Count Courses 0. Exit ======================================
```

21. Step 18 – Input Validation

• Do not allow empty student name.

• Validate email format as a basic check.

• Do not accept negative age.

• Do not accept negative course fees.

• Handle invalid student/course IDs.

• Handle SQLException without crashing the menu.

22. Final Project Requirements

■ Use exactly 2 database tables: student and course.

■ Do not use JOIN.

■ Use DTO classes for data.

■ Use DAO interfaces.

■ Use DAO implementation classes.

■ Use ConnectionFactory.

■ Use PreparedStatement.

■ Use ResultSet.

■ Implement CRUD for both tables.

■ Implement at least 5 search/filter operations.

■ Create a menu-driven main program.

■ Do not write SQL directly inside the main class.

23. Student Project Tasks

1. Create database and both tables.

2. Create project packages.

3. Configure MySQL JDBC driver.

4. Create ConnectionFactory.

5. Create Student DTO.

6. Create Course DTO.

7. Create StudentDAO.

8. Create CourseDAO.

9. Create StudentDAOImpl.

10. Create CourseDAOImpl.

11. Test Student CRUD.

12. Test Course CRUD.

13. Add search operations.

14. Add sorting operations.

15. Add count operations.

16. Create final menu.

17. Test all menu options.

24. Final Testing Checklist

Test	Status
Database connection works	■
Student INSERT works	■

Test	Status
Student SELECT works	■
Student UPDATE works	■
Student DELETE works	■
Student search works	■
Course INSERT works	■
Course SELECT works	■
Course UPDATE works	■
Course DELETE works	■
Course search works	■
Invalid input handled	■
Complete menu tested	■

Important: Complete one step successfully before moving to the next step. Do not copy the complete project at once. The objective is to understand how DTO, DAO, DAO Implementation, ConnectionFactory and JDBC work together.